



SCAN_TO_VISIT_REPO

[SYSTEM :: THREAT_DETECTION_ENGINE]

ZERO CONTEXT

AI-Powered Intrusion Prevention System

CONTINUOUS_AUTHENTICATION //BEHAVIORAL_BIOMETRICS //ZERO_TRUST

github.com/A23894843/Zero_Context

Submitted in partial fulfilment of the academic requirements for the successful completion of the Third Year of the Bachelor of Technology

SUBMITTED_BY

Abhinandan

Enrollment No. 236301015

DEPARTMENT

Computer Science and Engineering

Specialization: Cybersecurity

INSTITUTION

Gurukula Kangri (Deemed to be University)

Haridwar, Uttarakhand

ACADEMIC_YEAR

2025 - 2026

STATUS: SOLO_ENGINEERED

ZERO CONTEXT

AI-Powered Intrusion Prevention System

Project Synopsis

https://github.com/A23894843/Zero_Context

Submitted By: Abhinandan (236301015)

Department of Computer Science and Engineering
Gurukula Kangri (Deemed to be University), Haridwar, Uttarakhand
2025-2026

1 Introduction

Modern cybersecurity systems overwhelmingly rely on perimeter defense and one-time authentication: once a user or session is verified, the system implicitly trusts every subsequent action. This assumption breaks down the moment an attacker steals credentials, hijacks a session, or deploys an automated payload such as a Hak5 Rubber Ducky or a Remote Access Trojan (RAT). **Zero Context** addresses this gap by extending Zero Trust principles into *Continuous Authentication*: it constantly verifies that a real human, not a script, is operating the machine, by analyzing the organic biometric signature of mouse and keyboard input in real time.

2 Problem Statement

Existing endpoint security tools are unable to reliably distinguish human input from synthetic or automated input once a session is already authenticated. This creates four critical gaps:

- Signature-based Antivirus/EDR tools cannot flag legitimate OS commands executed by an attacker script.
- Cloud-based anomaly detection introduces network latency, giving attackers a window to act before a response is triggered.
- Deep learning-based detection is often too resource-intensive for continuous, on-device background monitoring.
- Most systems alert a Security Operations Center (SOC) but take no automated action, allowing an attack to continue unchecked.

3 Objectives

1. Engineer a low-level C++ daemon that reads raw hardware input events directly from the Linux kernel with minimal latency.

2. Build a highly concurrent, asynchronous Python backend using `asyncio` and Unix Domain Sockets for efficient telemetry ingestion.
3. Train and deploy unsupervised machine learning models — a PyTorch Autoencoder and a Scikit-Learn Isolation Forest — capable of sub-millisecond inference.
4. Implement a non-blocking MySQL/MariaDB persistence layer using `aiomysql` that logs threats without stalling the sensor stream.
5. Trigger automated, real-time mitigation (D-Bus session termination) the moment a critical anomaly is detected.
6. Validate the system under extreme load to demonstrate real-world, enterprise-grade viability.
7. Package the system for reproducible deployment using Docker and Docker Compose.

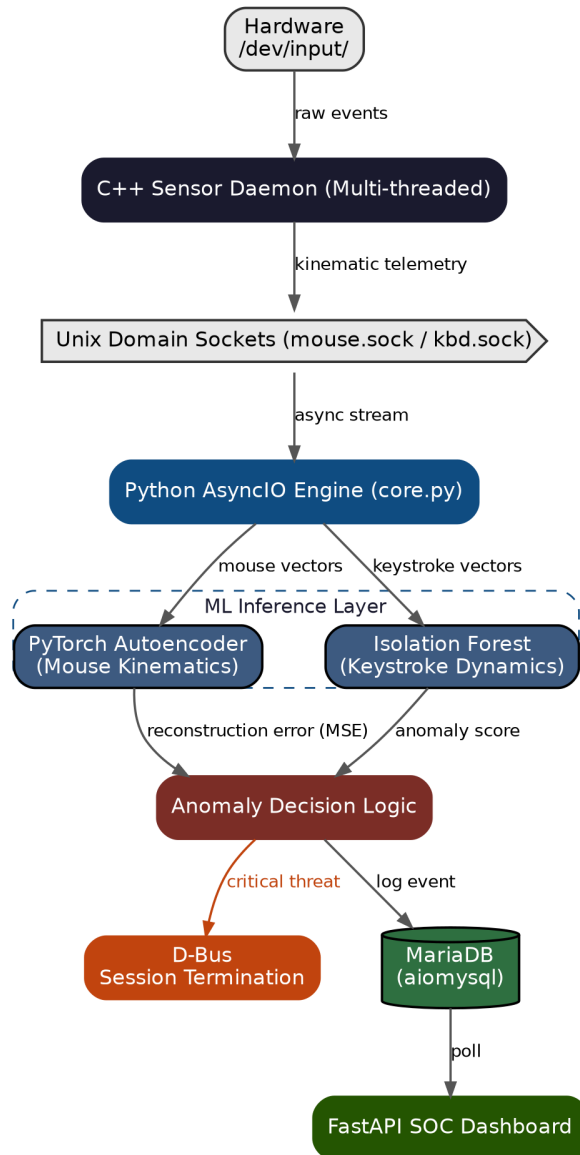
4 Proposed Methodology

Zero Context is built as a decoupled, multi-stage pipeline so that each layer can be optimized independently: a low-level capture layer in C++, an asynchronous inference layer in Python, a persistence layer in MariaDB, and a presentation layer in FastAPI.

4.1 System Architecture

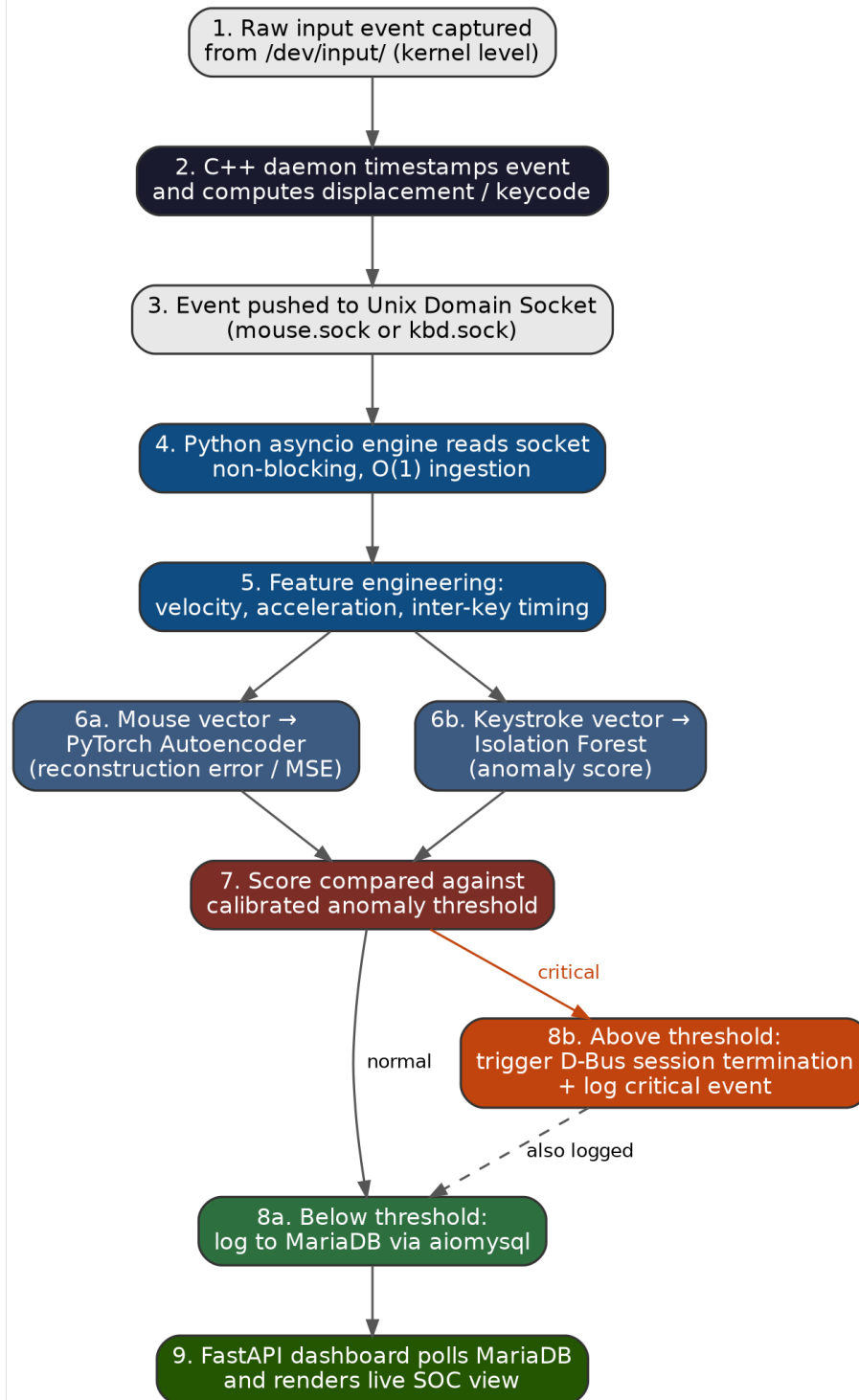
Raw input events are captured at the kernel level and streamed with minimal overhead to the inference engine, which scores each event using two purpose-built ML models before deciding whether to log or actively intervene.

Zero Context — High-Level System Architecture



4.2 Data Flow

Zero Context — End-to-End Data Flow



4.3 Database Design

A single, write-optimized table persists all triggered alerts for later SOC review and dashboard visualization.

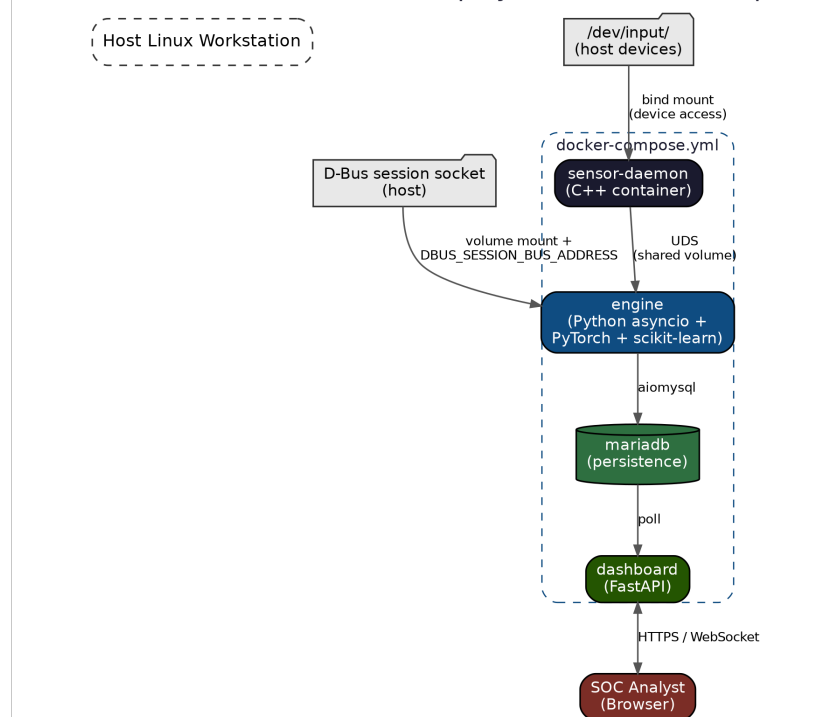
Zero Context — Threat Alerts Schema

threat_alerts	
alert_id	INT, PK, AUTO_INCREMENT
timestamp	DOUBLE, NOT NULL
sensor_type	VARCHAR(64), NOT NULL
anomaly_score	DOUBLE, NOT NULL
description	TEXT, NOT NULL
created_at	TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

4.4 Deployment Strategy

The full stack is containerized using Docker Compose, with careful host-device and D-Bus socket mounting so that a hardware-level sensor daemon can still function correctly inside a container.

Zero Context — Containerized Deployment (Docker Compose)



5 Tools and Technologies

Layer	Technology
Sensor / Capture	C++17, Linux /dev/input/
Inter-Process Communication	Unix Domain Sockets
Inference Engine	Python 3, asyncio
Machine Learning	PyTorch (Autoencoder), Scikit-Learn (Isolation Forest)
Persistence	MariaDB, aiomysql
Mitigation	D-Bus
Dashboard	FastAPI
Deployment	Docker, Docker Compose

6 Expected Outcome

The completed system is expected to demonstrate that continuous, ML-driven behavioral authentication can run silently on commodity hardware without degrading system performance, while providing meaningfully faster and more automated threat response than traditional EDR tools. Preliminary extreme-load testing has already validated throughput of **18.8 million events per second** with a median inference latency (P50) of **0.0023 ms**, a **100%** detection rate on synthetic payloads, and an **F1-score of 0.971** for keystroke anomaly detection.

7 Scope

The current implementation targets Linux-based workstations and servers, leveraging /dev/input/ for telemetry capture and the D-Bus session bus for mitigation. Future iterations may extend mitigation to network-level containment (iptables), add continuous model retraining, and incorporate process-context awareness for richer forensic reporting.